Abdelmalek Essaâdi University

Faculty of Sciences and Technologies Tangier

Master of Science and Technology
IT Security and Big Data

Lab 3 - Snapshots, Clones & Linked Clones

Analyse du mecanisme COW et gestion des snapshots sous ESXi

Student : Azzaoui Mohamed Omar
Instructor : Mohamed BEN AHMED

Academic Year : 2025–2026

Introduction

Ce travail porte sur la gestion des snapshots VMware dans un environnement ESXi autonome. Le but n'etait pas seulement d'executer les commandes demandees, mais surtout de comprendre le comportement reel du stockage lors de la creation, de la restauration et de la consolidation des snapshots. Dans notre cas pratique, la machine virtuelle cible est **ubuntu** (VMID 3), hebergee sur le datastore **ubuntu** de type VMFS-6. Cette VM a servi de machine de travail pour la creation d'un contenu test dans `/tmp/test.txt`, puis pour l'observation de la chaine des snapshots.

L'analyse montre que VMware n'effectue pas une copie complete du disque lors d'un snapshot. Le disque de base est fige en lecture seule, puis toutes les nouvelles ecritures sont redirigees vers des fichiers differentiels. Sur notre infrastructure, ces fichiers ne portent pas le suffixe classique `-delta.vmdk` mentionne dans le support, mais le format `sesparse`, ce qui est coherent avec un environnement VMFS-6. Cette difference est importante car elle montre que le principe reste identique, meme si la representation physique des deltas varie selon la version de stockage utilisee.

3.1 Comprendre les snapshots VMware

Le mecanisme de snapshot repose sur le principe **COW** (Copy-On-Write). Lorsqu'un snapshot est cree :

- le disque de base est conserve comme reference stable ;
- un descripteur de snapshot est cree ;
- les nouvelles ecritures sont stockees dans un fichier differentiel ;
- la machine continue de fonctionner, mais l'etat courant depend d'une chaine logique composee du disque de base et des deltas successifs.

Dans notre cas, la chaine observee est la suivante :

`ubuntu-flat.vmdk` → `ubuntu-000001-sesparse.vmdk` → `ubuntu-000002-sesparse.vmdk`

Cela signifie que le snapshot `snap-01-avant-apache` constitue la premiere couche de modification, tandis que `snap-02-apres-modification` represente l'etat actif le plus recent au moment de l'observation.

Fichier	**Rôle observé**	**Exemple dans notre cas**
`.vmdk` (base)	Descripteur du disque principal	`ubuntu.vmdk`
`-flat.vmdk`	Données du disque principal, utilisées comme base de référence	`ubuntu-flat.vmdk`
`-000001.vmdk`	Descripteur du premier snapshot	`ubuntu-000001.vmdk`
`-000001-sesparse.vmdk`	Première couche différentielle issue du snapshot 1	`ubuntu-000001-sesparse.vmdk`
`-000002.vmdk`	Descripteur du deuxième snapshot	`ubuntu-000002.vmdk`
`-000002-sesparse.vmdk`	Deuxième couche différentielle, active au moment de la capture	`ubuntu-000002-sesparse.vmdk`
`.vmsd`	Métadonnées de la chaîne de snapshots	`ubuntu.vmsd`

3.2 Etape 1 - Creation des snapshots

Preparation du contenu test dans la VM

Avant de creer les snapshots, un contenu simple et traçable a ete cree dans la VM **ubuntu**. Le fichier `/tmp/test.txt` sert de trace d'etat afin de verifier ensuite le comportement du revert.

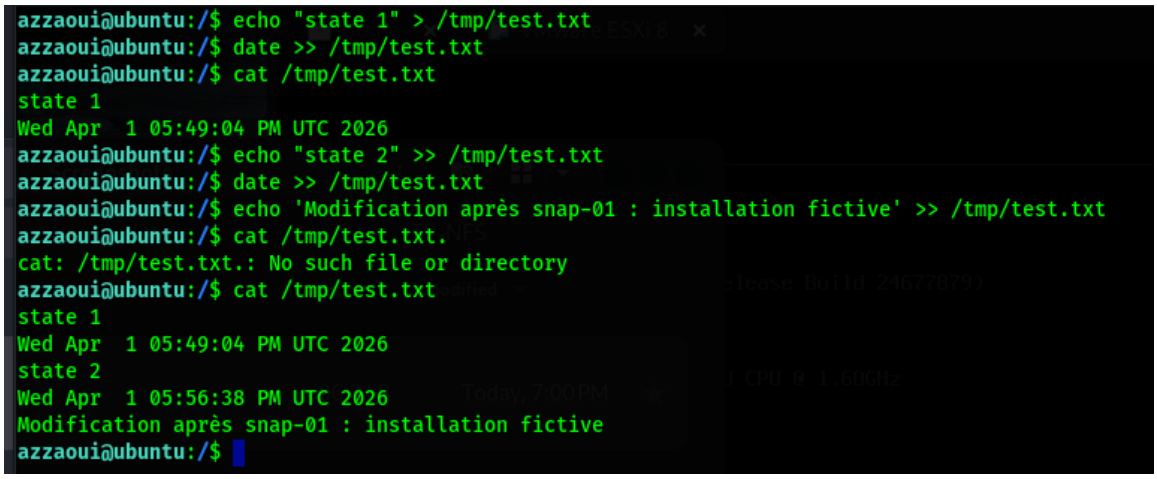

FIGURE 1 – Preparation du contenu test dans la VM `ubuntu` - fichier `testtxt.png`. La capture montre d'abord la creation de l'etat initial (`state 1` et horodatage), puis l'ajout de `state 2` et d'une ligne supplementaire apres le premier snapshot. On observe egalement une petite erreur de frappe dans une commande `cat`, corrigee juste apres ; le resultat final reste coherent et prouve que la VM a bien ete modifiee entre les deux snapshots.

Cette capture est importante, car elle prouve que le second snapshot n'est pas pris sur un etat identique au premier. Autrement dit, il existe une difference fonctionnelle reelle entre les deux points de restauration.

Creation des snapshots depuis ESXi

Les snapshots ont ete crees via la commande `vim-cmd`. La VM cible est `ubuntu` avec l'identifiant `VMID 3`. Deux snapshots ont ete crees successivement :

- `snap-01-avant-apache` ;
- `snap-02-apres-modification`.

```
[root@localhost:~] vim-cmd vmsvc/snapshot.create 3 "snap-01-avant-apache" \
> "Etat initial avant modification" 0 0
Create Snapshot:
[root@localhost:~] vim-cmd vmsvc/snapshot.get 3
Get Snapshot:
|-ROOT
--Snapshot Name        : snap-01-avant-apache
--Snapshot Id        : 1
--Snapshot Desciption  : Etat initial avant modification
--Snapshot Created On  : 4/1/2026 17:56:4
--Snapshot State       : powered off
[root@localhost:~] vim-cmd vmsvc/snapshot.create 3 "snap-02-apres-modification" \
> "Apres modification du systeme" 0 0
Create Snapshot:
[root@localhost:~] vim-cmd vmsvc/snapshot.get 3
Get Snapshot:
|-ROOT
--Snapshot Name        : snap-01-avant-apache
--Snapshot Id        : 1
--Snapshot Desciption  : Etat initial avant modification
--Snapshot Created On  : 4/1/2026 17:56:4
--Snapshot State       : powered off
--|-CHILD
----Snapshot Name        : snap-02-apres-modification
----Snapshot Id        : 2
----Snapshot Desciption  : Apres modification du systeme
----Snapshot Created On  : 4/1/2026 17:57:24
----Snapshot State       : powered off
```

FIGURE 2 – Creation et enumeration des snapshots depuis le shell ESXi - fichier `esxi-snapshots.png`. La premiere partie montre la commande de creation du snapshot 1 avec la description « Etat initial avant modification ». La seconde partie montre la creation du snapshot 2, puis l'affichage de l'arborescence obtenue. L'analyse de la sortie confirme l'existence d'un noeud `ROOT`, du snapshot parent `snap-01-avant-apache` et de son enfant `snap-02-apres-modification`. Cette hierarchie prouve que le deuxieme snapshot depend du premier et que VMware construit bien une chaine ordonnee d'etats.

Observation des fichiers differentiels

L'inspection du repertoire de la VM sur le datastore `ubuntu` montre la transformation physique de l'etat logique des snapshots en fichiers de travail.

```
[root@localhost:~] du -sh /vmfs/volumes/ubuntu/ubuntu/*sesparse*
23.0M   /vmfs/volumes/ubuntu/ubuntu/ubuntu-000001-sesparse.vmdk
19.0M   /vmfs/volumes/ubuntu/ubuntu/ubuntu-000002-sesparse.vmdk
[root@localhost:~]
```

FIGURE 3 – Mesure de la taille des fichiers differentiels `sesparse` - fichier `sesparseeq-delta.png`. Contrairement au support de cours qui mentionne `*-delta.vmdk`, notre environnement VMFS-6 utilise des fichiers `*-sesparse.vmdk`. Les tailles observees (environ 23 Mo pour `ubuntu-000001-sesparse.vmdk` et 19 Mo pour `ubuntu-000002-sesparse.vmdk`) montrent clairement que VMware ne recopie pas l'ensemble du disque de 45 Go ; seules les modifications successives sont stockees. C'est une demonstration concrete du mecanisme Copy-On-Write.

L'analyse des tailles est tres importante : un disque de base de 45 Go n'engendre

pas des snapshots de meme taille. Les deux deltas restent de taille reduite car ils ne contiennent que les blocs modifies apres chaque prise de snapshot.

3.3 Etape 2 - Restaurer un snapshot (Revert)

Le revert a ete realise depuis l'interface de gestion des snapshots. Le choix a porte sur le snapshot `snap-01-avant-apache`, ce qui signifie un retour a l'etat initial de la VM avant les modifications ulterieures.

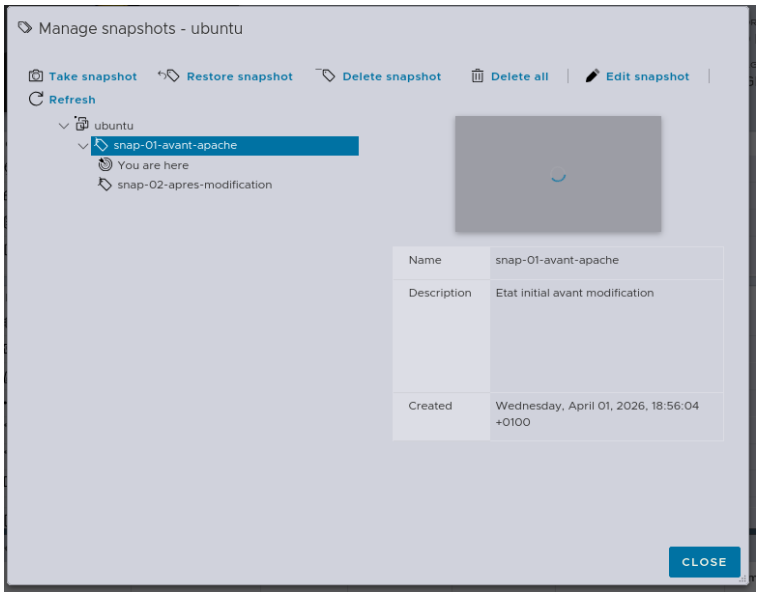

FIGURE 4 – Gestionnaire de snapshots et selection du point de restauration - fichier `snapshotrevertion.png`. L'interface affiche la chaine `ubuntu` → `snap-01-avant-apache` → `snap-02-apres-modification`. La mention « You are here » situe l'etat courant avant le revert, tandis que les details du snapshot 1 apparaissent a droite. Cette capture permet de comprendre visuellement qu'un revert ne consiste pas a copier un fichier, mais a repositionner l'etat actif de la VM dans la chaine des snapshots.

Apres le revert, le fichier `/tmp/test.txt` a ete verifie a l'interieur de la VM.

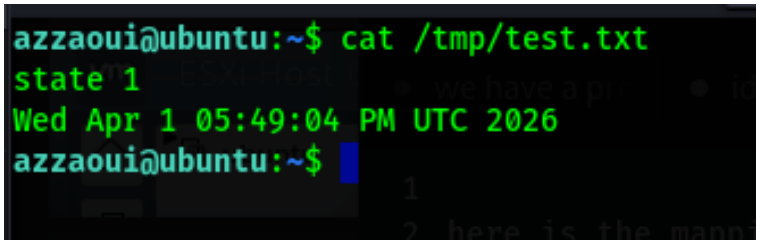

FIGURE 5 – Verification du contenu du fichier apres restauration - fichier `testtxtAFTER.png`. Le fichier `/tmp/test.txt` ne contient plus que `state 1` et son horodatage initial. Les ajouts effectues apres le premier snapshot ne sont plus presents. Cette verification valide le principe du revert : toutes les modifications posterieures au snapshot cible sont abandonnees.

Cette observation confirme le comportement destructif du revert. En revenant a `snap-01-avant-apac`

les modifications associees a `snap-02-apres-modification` disparaissent de l'etat actif de la machine.

3.4 Etape 3 - Suppression et consolidation des snapshots

Apres validation du revert, l'ensemble des snapshots a ete supprime au moyen de la commande `snapshot.removeall`. Cette operation ne se limite pas a effacer une liste logique de snapshots ; elle declenche une **consolidation**, c'est-a-dire la fusion des blocs utiles dans l'etat de disque final et la suppression des couches differencielles devenues inutiles.

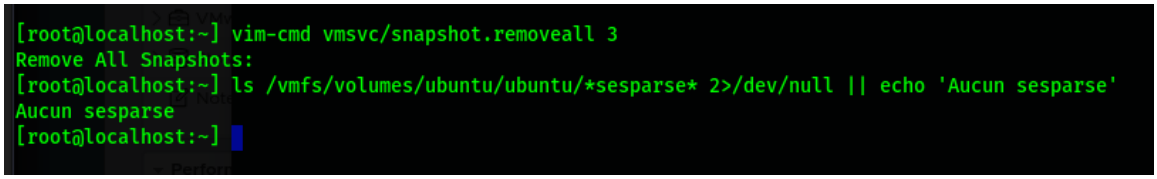

FIGURE 6 – Suppression de tous les snapshots et verification finale - fichier `Remove-all-snapshots.png`. La commande `vim-cmd vmsvc/snapshot.removeall 3` declenche la suppression complete de la chaine de snapshots. La verification immediate par `ls /vmfs/volumes/ubuntu/ubuntu/*sesparse* 2>/dev/null || echo 'Aucun sesparse'` retourne `Aucun sesparse`. Cette sortie demontre que la consolidation s'est terminee correctement et qu'il ne reste plus de couche differentielle associee a la VM.

Le resultat ci-dessus est l'un des points les plus importants du lab. Avant la consolidation, la VM reposait sur une chaine de couches successives ; apres `removeall`, la chaine est aplanie et le stockage redevient propre.

Apercu conceptuel - Full Clone et Linked Clone

La derniere partie du support de TP introduit la distinction entre **Full Clone** et **Linked Clone**. Dans notre environnement pratique, l'etude s'est concentree sur les snapshots et la consolidation, car ces mecanismes etaient directement observables et verifiables sur la VM `ubuntu`. Toutefois, il est utile de rappeler la difference conceptuelle entre les deux types de clonage :

- **Full Clone** : copie totalement independante de la VM source. Le clone possede ses propres fichiers VMDK et peut fonctionner sans dependre de la machine d'origine.
- **Linked Clone** : clone dependant d'un snapshot parent. Il reutilise le disque de base et ne stocke que les differences, ce qui economise de l'espace mais introduit une dependance structurelle a la VM source.

En pratique, le comportement du **Linked Clone** prolonge directement la logique du snapshot et du Copy-On-Write observee dans ce travail. Le **Full Clone**, au contraire, rompt cette dependance en recreant un disque autonome.

Conclusion

Ce TP a permis de verifier concretement le fonctionnement des snapshots VMware dans un environnement ESXi autonome. La sequence experimentale a ete la suivante :

creation d'un etat initial, creation d'un premier snapshot, modification de la VM, creation d'un deuxieme snapshot, observation des fichiers differenciels, restauration vers le premier snapshot, puis suppression complete et consolidation.

Les captures et les commandes executees montrent plusieurs points essentiels :

— un snapshot n'est pas une copie complete du disque, mais une couche differentielle ;
— sous VMFS-6, les differenciels apparaissent sous forme de fichiers `sesparse` ;
— le revert repositionne l'etat actif de la VM sur un point precis de la chaine ;
— la consolidation supprime les couches temporaires et restitue un stockage propre ;
— la logique des snapshots constitue la base conceptuelle necessaire pour comprendre ensuite la difference entre Full Clone et Linked Clone.

Ainsi, ce travail valide a la fois la maitrise des commandes de gestion des snapshots et la comprehension du modele de stockage sous-jacent.